



Machine Learning in Production Model Testing beyond Accuracy

Learning Goals

- Curate validation datasets for assessing model quality, covering subpopulations and capabilities as needed
- Explain the oracle problem and how it challenges testing of software and models
- Use invariants to check partial model properties with automated testing
- Select and deploy automated infrastructure to evaluate and monitor model quality



Curating Validation Data & Input Slicing



Breakout Discussion

Write a few tests for the following program:

```
def nextDate(year: Int, month: Int, day: Int) = ...
```

A test may look like:

```
assert nextDate(2021, 2, 8) == (2021, 2, 9);
```

As a group, discuss how you select tests. Discuss how many tests you need to feel confident.

Post answer to #lecture tagging group members in Slack using template:

Selection strategy: ...

Test quantity: ...

Software Test Case Design

Opportunistic/exploratory testing: Add some unit tests, without much planning

Specification-based testing ("black box"): Derive test cases from specifications

- Boundary value analysis
- Equivalence classes
- Combinatorial testing
- Random testing

Structural testing ("white box"): Derive test cases to cover implementation paths

- Line coverage, branch coverage
- Control-flow, data-flow testing, MCDC, ...

Test execution usually automated, but can be manual too; automated generation from specifications or code possible

Example: Boundary Value Testing

Analyze the specification, not the implementation!

Key Insight: Errors often occur at the boundaries of a variable value

For each variable select (1) minimum, (2) min+1, (3) medium, (4) max-1, and (5) maximum; possibly also invalid values min-1, max+1

Example: `nextDate(2015, 6, 13) = (2015, 6, 14)`

- **Boundaries?**

Example: Equivalence classes

Idea: Typically many values behave similarly, but some groups of values are different

Equivalence classes derived from specifications (e.g., cases, input ranges, error conditions, fault models)

Example `nextDate(2015, 6, 13)`

- leap years, month with 28/30/31 days, days 1-28, 29, 30, 31

Pick 1 value from each group, combine groups from all variables

Thought experiment

```
/** Compute the price of a bus ride:  
 * - Children under 2 ride for free, children under 18 and  
 *   senior citizen over 65 pay half, all others pay the  
 *   full fare of $3.  
 * - On weekdays, between 7am and 9am and between 4pm and  
 *   7pm a peak surcharge of $1.5 is added.  
 * - Short trips under 5min during off-peak time are free.*/  
def busTicketPrice(age: Int,  
                    datetime: LocalDateTime,  
                    rideTime: Int)
```

suggest test cases based on boundary value analysis and equivalence class testing

Selecting Validation Data for Model Quality?

- Validation data should reflect usage data
- Be aware of data drift (face recognition during pandemic, new patterns in credit card fraud detection)
- "*Out of distribution*" predictions often low quality (it may even be worth to detect out of distribution data in production, more later)

(note, similar to requirements validation: did we hear all/representative stakeholders)

Not All Inputs are Equal: Frequent Cases



"Call mom" "What's the weather tomorrow?" "Add asafetida to my shopping list"

Not All Inputs are Equal: Edge Cases



Not All Inputs are Equal

some random mistakes vs rare but biased mistakes?

- A system to detect when somebody is at the door that never works for people under 5ft (1.52m)
- A spam filter that deletes alerts from banks
- Technology from Amazon, Apple, Google, IBM and Microsoft misidentified 35 percent of words from people who were black. White people fared much better. -- [NYTimes March 2020](#)

How do you identify Important Inputs?

(We already hinted so...)



Identify Important Inputs

Curate **Validation Data** for Specific Problems and Subpopulations:

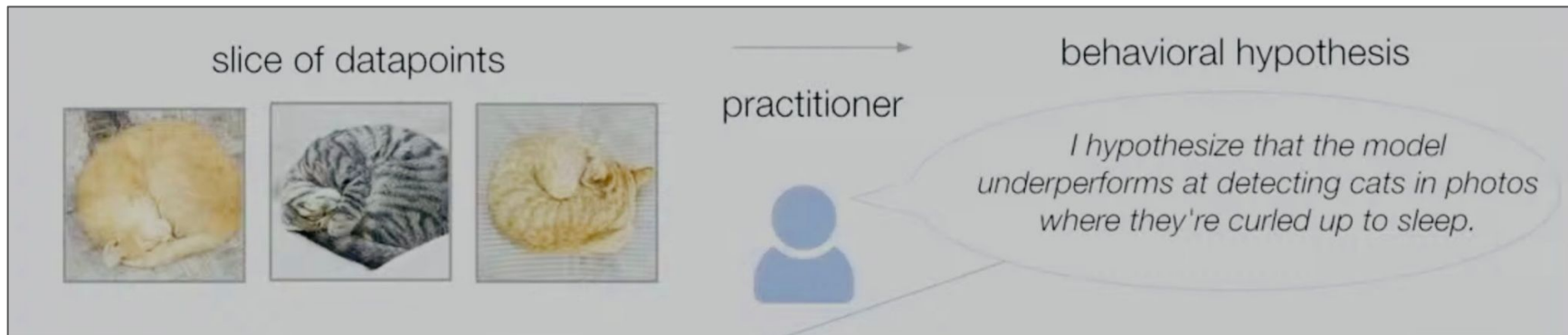
- Important inputs ("call mom") -- expect very high accuracy
 - closest equivalent to **unit tests**
- Different subpopulations (e.g., accents) -- expect comparable accuracy
- Challenging cases or stretch goals -- accept lower accuracy

Derive from requirements, experts, user feedback, expected problems etc. Think *specification-based testing*.

Access to Important Inputs: Partitioning

- Guide testing by identifying groups and analyzing accuracy of subgroups
 - Often for fairness: gender, country, age groups, ...
 - Possibly based on business requirements or cost of mistakes
- Slice test data by population criteria, also evaluate interactions
- Identifies problems and plan mitigations, e.g., enhance with more data for subgroup or reduce confidence

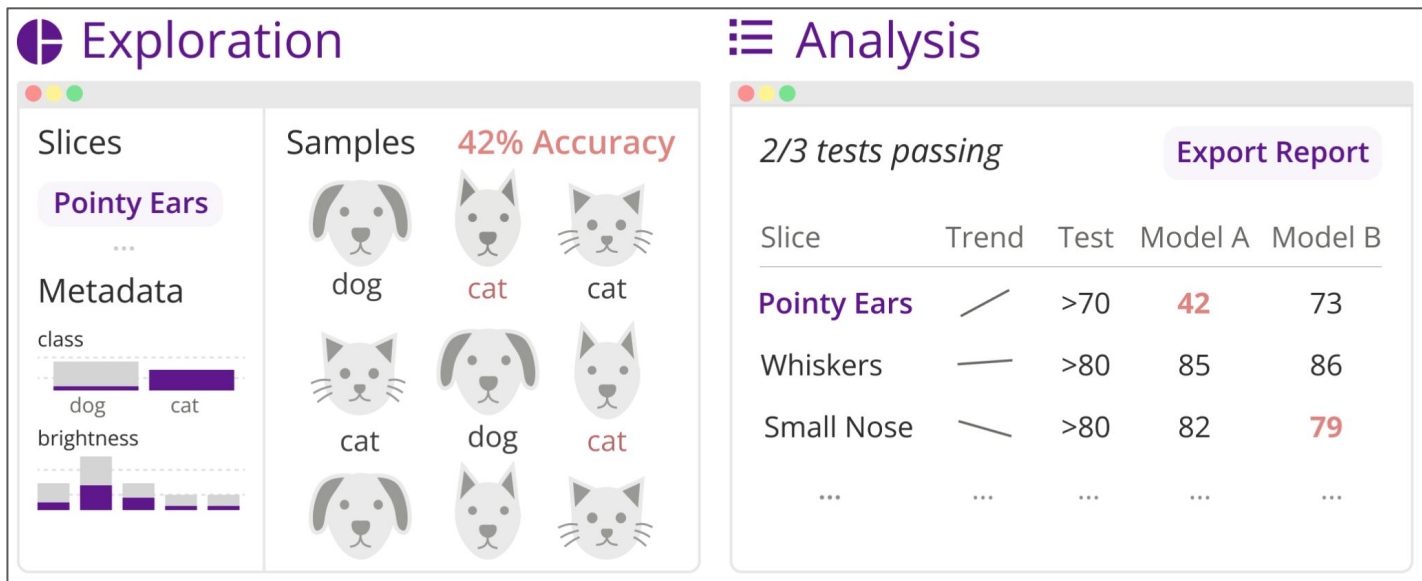
Input Partitioning Example



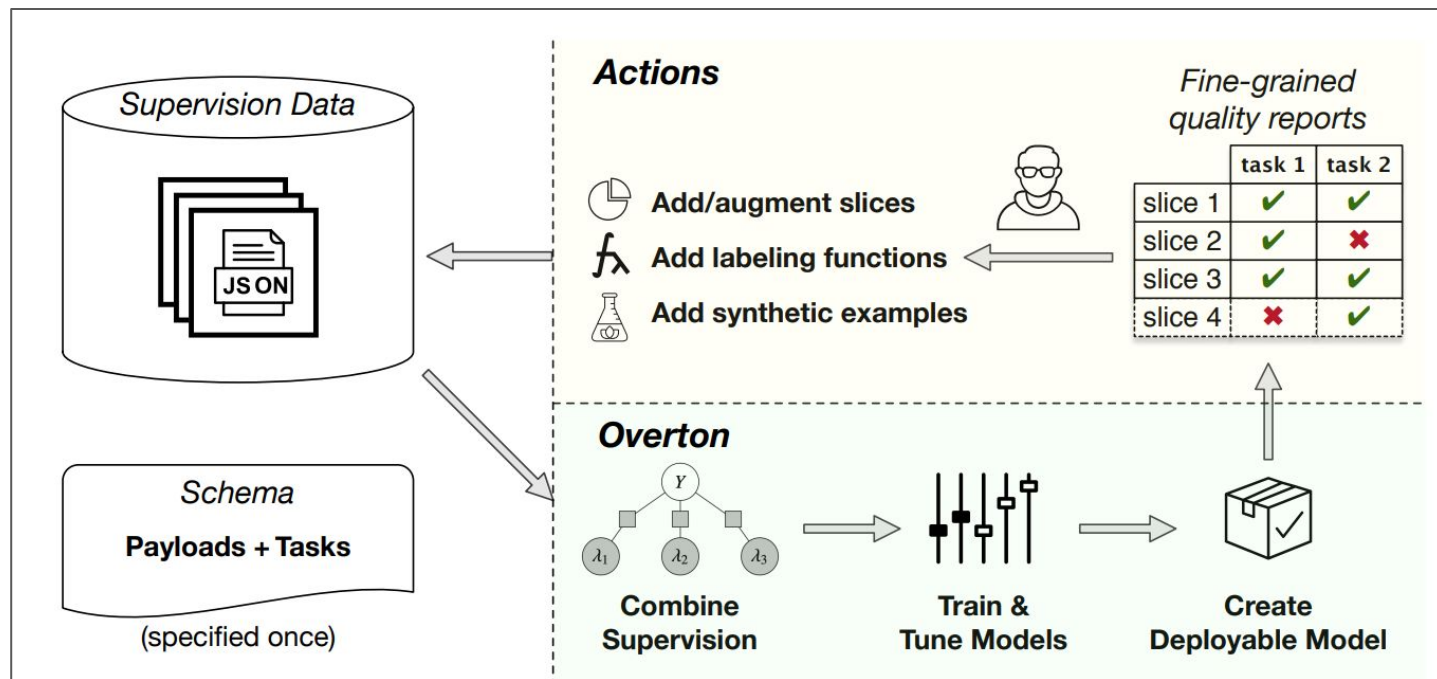
Slice and hypothesis on model's cat recognition behavior

Input Partitioning Example

Multiple slices on image recognition, and model comparison



Example: Model Impr. at Apple (Overton)



Example: Model Impr. at Apple (Overton)

- Focus engineers on creating training and validation data, not on model search (AutoML)
- Flexible infrastructure to slice telemetry data to identify underperforming subpopulations -> focus on creating better training data (better, more labels, in semi-supervised learning setting)

Input Partitioning Discussion

How to slice evaluation data for cancer prognosis?



Input Partitioning Discussion

How to slice evaluation data for Excel AI?



Behavioral Testing (Capabilities)



Testing Capabilities

Core idea: Define "capabilities" that the model should grasp in order to do a test well, and create targeted test cases for those capabilities.

Negation	<i>MFT</i> : Negated negative should be positive or neutral	18.8	54.2	29.4	13.2	2.6	The food is not poor. pos or neutral It isn't a lousy customer service. pos or neutral
	<i>MFT</i> : Negated neutral should still be neutral	40.4	39.6	74.2	98.4	95.4	This aircraft is not private. neutral This is not an international flight. neutral
	<i>MFT</i> : Negation of negative at the end, should be pos. or neut.	100.0	90.4	100.0	84.8	7.2	I thought the plane would be awful, but it wasn't. pos or neutral I thought I would dislike that plane, but I didn't. pos or neutral
	<i>MFT</i> : Negated positive with neutral content in the middle	98.4	100.0	100.0	74.0	30.2	I wouldn't say, given it's a Tuesday, that this pilot was great. neg I don't think, given my history with airplanes, that this is an amazing staff. neg
	<i>MFT</i> : Author sentiment is more important than of others	45.4	62.4	68.0	38.8	30.0	Some people think you are excellent, but I think you are nasty. neg Some people hate you, but I think you are exceptional. pos

Where do capabilities come from?

Ribeiro, Marco Tulio, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. "[Beyond Accuracy: Behavioral Testing of NLP Models with CheckList](#)." In Proceedings ACL, p. 4902–4912. (2020).

Capabilities and where they come from

In the paper we listed a bunch of standard, "shared" capabilities across NLP tasks -- e.g. whatever model it is, it should be able to handle negation, synonyms, etc., just that the model should react in different ways.

Should be more use-case specific and from domain knowledge!

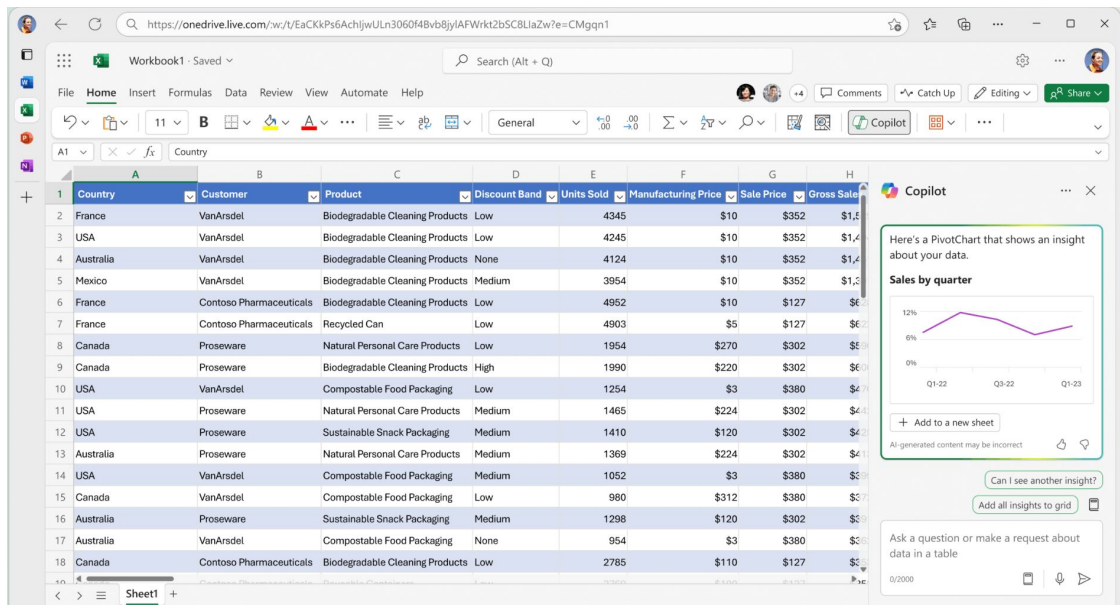
Capabilities	Descriptions
Vocab/POS	important words or word types for the task.
Named entities	appropriately understanding named entities.
Nagation	understand the negation words.
Taxonomy	synonyms, antonyms, etc.
Robustness	to typos, irrelevant changes, etc.
Coreference	resolve ambiguous pronouns, etc.
Fairness	not biasing towards certain gender/race groups.
Semantic Role Labeling	understanding roles such as agent, object, etc.
Logic	handle symmetry, consistency, and conjunctions.
Temporal	understand order of events.

Ribeiro, Marco Tulio, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. "[Beyond Accuracy: Behavioral Testing of NLP Models with CheckList](#)." In Proceedings ACL, p. 4902–4912. (2020).

Strategies for identifying capabilities

- Analyze common mistakes (e.g., classify past mistakes in cancer prognosis)
- Use existing knowledge about the problem (e.g., linguistics theories)
- Observe humans (e.g., how do radiologists look for cancer)
- Derive from requirements (e.g., fairness)
- Causal discovery from observational data?

Examples of Capabilities



What could be capabilities of the Excel AI?

Capabilities: common paradigm in LLM testing



MMLU (Massive Multitask Language Understanding): 16,000 multiple-choice questions across 57 subjects → capabilities (biology, US history, law.)

- Aggregate accuracy hides variation.

HumanEval: 164 hand-written Python programming problems (function signature + docstring)

- Tested via execution against hidden unit tests (simulation-based testing!)

Both benchmarks operationalize "what should this model be able to do", slicing by subject/problem type.

Capabilities vs Specifications vs Slicing

Capabilities are partial specifications of expected behavior (not expected to always hold)

Some capabilities correspond to slices of existing test data, for others we may need to **create new data**

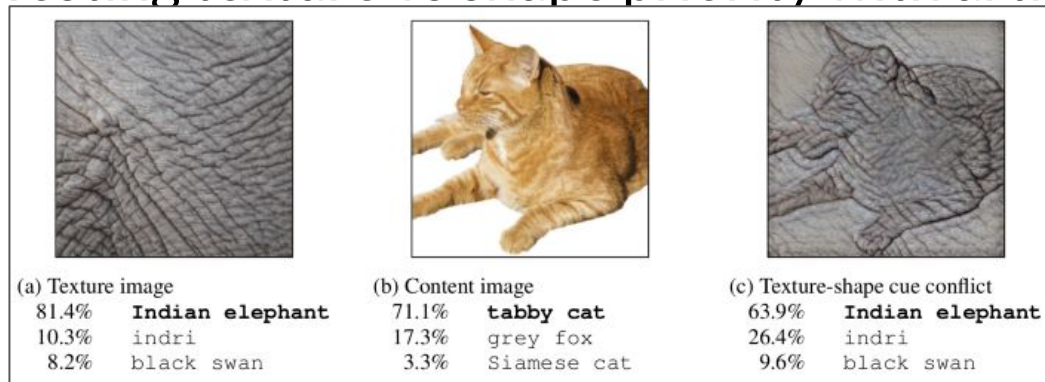
Generating Test Data for Capabilities

Idea 1: Domain-specific generators

Testing *negation* in sentiment analysis with template:

I {NEGATION} {POS_VERB} the {THING}.

Testing texture vs shape priority with artificial generated images:



Generating Test Data for Capabilities

Idea 2: Mutating existing inputs

- Testing *synonyms* in sentiment analysis by replacing words with synonyms, keeping label
- Testing *robust against noise and distraction* add and false is not true or random URLs to text

Robust.	INV: Add randomly generated URLs and handles to tweets	9.6	13.4	24.8	11.4	7.4	@JetBlue that selfie was extreme. @pi9QDK INV @united stuck because staff took a break? Not happy 1K.... https://t.co/PWK1jb INV
	INV: Swap one character with its neighbor (typo)	5.6	10.2	10.4	5.2	3.8	@JetBlue → @JeBtlue I cri INV @SouthwestAir no thanks → thakns INV
NER	INV: Switching locations should not change predictions	7.0	20.8	14.8	7.6	6.4	@JetBlue I want you guys to be the first to fly to # Cuba → Canada... INV @VirginAmerica I miss the #nerdbird in San Jose → Denver INV
	INV: Switching person names should not change predictions	2.4	15.1	9.1	6.6	2.4	...Airport agents were horrendous. Sharon → Erin was your saviour INV @united 8602947, Jon → Sean at http://t.co/58tuTgli0D , thanks. INV

Generating Test Data for Capabilities

Idea 3: Crowd-sourcing test creation

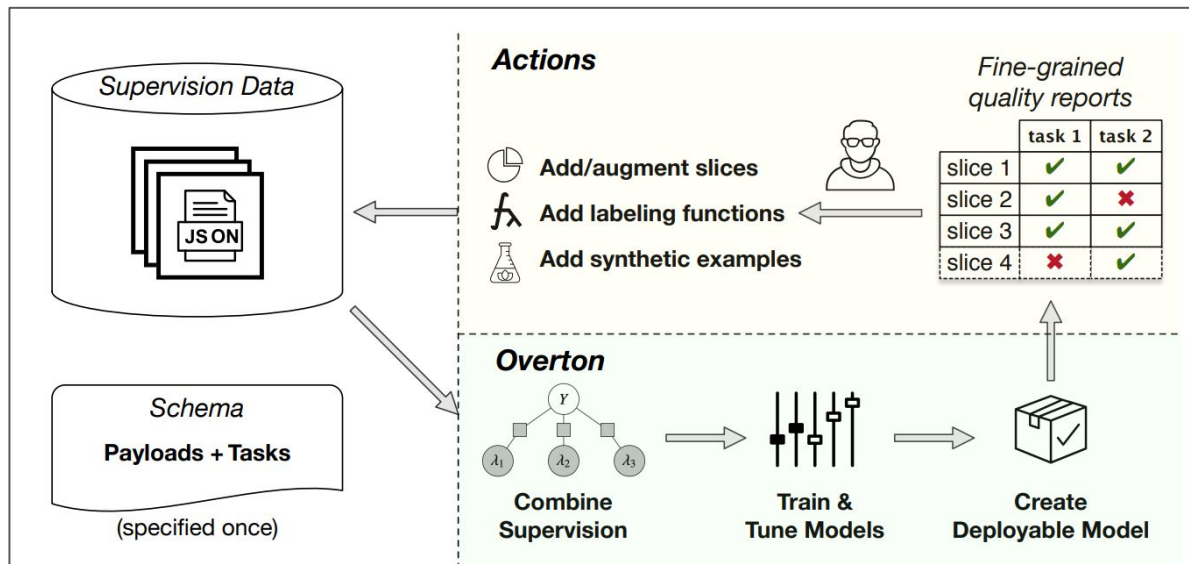
- Testing *sarcasm* in sentiment analysis: Ask humans to minimally change text to flip sentiment with sarcasm
- Testing *background* in object detection: Ask humans to take pictures of specific objects with unusual backgrounds

Recasting <i>fact</i> as <i>hoped for</i>	The world of Atlantis, hidden beneath the earth's core, is fantastic The world of Atlantis, hidden beneath the earth's core is supposed to be fantastic
Suggesting sarcasm	thoroughly captivating thriller-drama, taking a deep and realistic view thoroughly mind numbing "thriller-drama", taking a "deep" and "realistic" (who are they kidding?) view
Inserting modifiers	The presentation of simply Atlantis' landscape and setting The presentation of Atlantis' predictable landscape and setting

Generating Test Data for Capabilities

Idea 4: Slicing test data

Testing *negation* in sentiment analysis by finding sentences containing 'not'



Generating Test Data for Capabilities

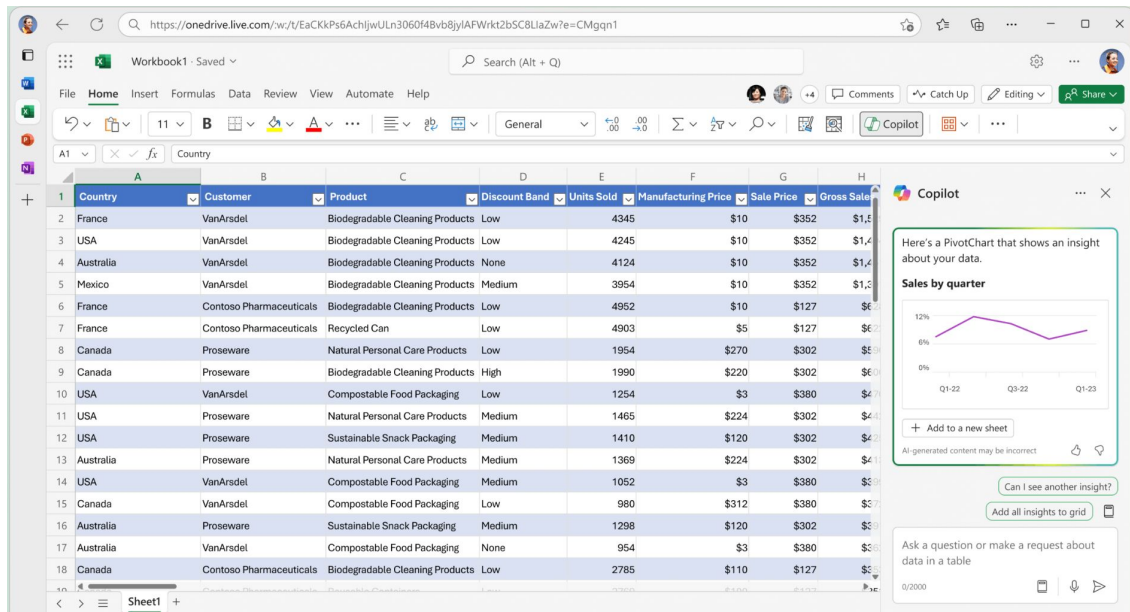
Idea 5: Directly synthesize data using LLMs

This can give you "pseudo" labeled data for tasks/capabilities largest models are known to have. Known as "distillation"

Table 1: Prompt template for the NYT news dataset.

Method	Prompt
SimPrompt	Suppose you are a news writer. Please generate a {topic-class} news in NYT.
AttrPrompt	Suppose you are a news writer. Please generate a {topic-class} news in NYT following the requirements below: 1. Should focus on {subtopic}; 2. Should be in length between {length:min-words} and {length:max-words} words; 3. The writing style of the news should be {style}; 4. The location of the news should be in {location}.

Generating Test Data for Capabilities



How to generate test data for capabilities of the Excel AI?

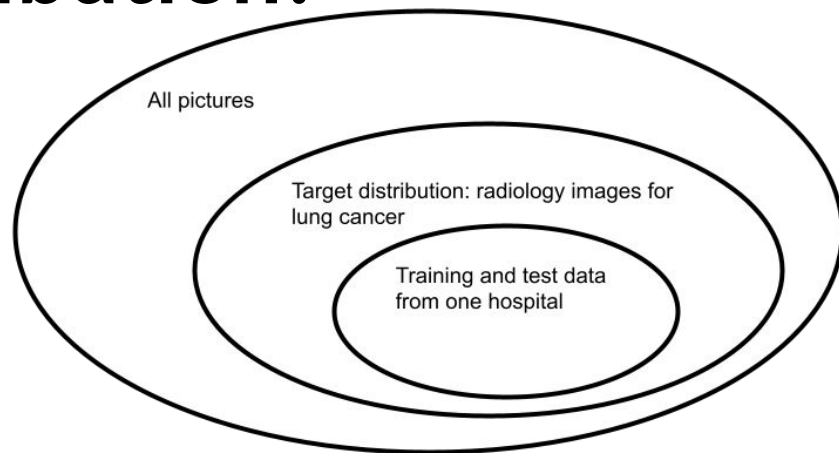
Testing vs Training Capabilities

- Dual insight for testing and training
- Strategies for curating test data can also help select training data
- Generate capability-specific training data to guide training (data augmentation)

Recasting <i>fact</i> as <i>hoped for</i>	The world of Atlantis, hidden beneath the earth's core, is fantastic The world of Atlantis, hidden beneath the earth's core is supposed to be fantastic
Suggesting sarcasm	thoroughly captivating thriller-drama, taking a deep and realistic view thoroughly mind numbing “thriller-drama”, taking a “deep” and “realistic” (who are they kidding?) view
Inserting modifiers	The presentation of simply Atlantis' landscape and setting The presentation of Atlantis' predictable landscape and setting

Why Augmentation: Generalization beyond Training Distribution?

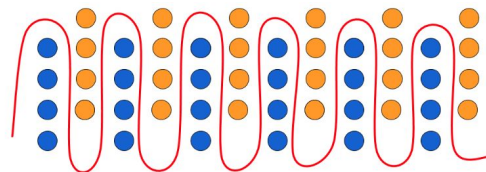
- Typically training and validation data from same distribution (i.i.d. assumption!)
- Many models can achieve similar accuracy
- Models that learn "right" abstractions possibly indistinguishable from models that use shortcuts
- Some models generalize better to other distributions not used in training
 - e.g., cancer images from other hospitals, from other populations
 - Drift and attacks, ...



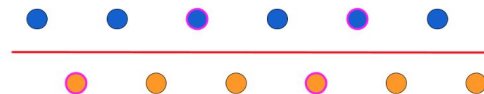
See discussion in D'Amour, Alexander, et al. "[Underspecification presents challenges for credibility in modern machine learning](https://arxiv.org/abs/2011.03395)." arXiv preprint arXiv:2011.03395 (2020).

Hypothesis: Capabilities may help

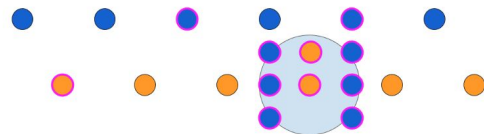
- Capabilities are "partial specifications", given beyond training data
- Encode domain knowledge of the problem
 - Capabilities are inherently domain specific
 - Curate capability-specific test data for a problem
- Testing for capabilities helps to distinguish models that use intended abstractions
- May help find models that generalize better



(a) A two-dimensional dataset that requires a complex decision boundary to achieve high accuracy.



(b) If the same data distribution is instead sampled with systematic gaps (e.g., due to annotator bias), a simple decision boundary *can perform well on i.i.d. test data* (shown outlined in pink).



(c) Since filling in all gaps in the distribution is infeasible, a *contrast set* instead fills in a local ball around a test instance to evaluate the model's decision boundary.

Matt Gardner et al. "[Evaluating Models' Local Decision Boundaries via Contrast Sets](#)." arXiv 2020.

On Terminology

- Test data curation is emerging as a very recent concept for testing ML components
- No consistent terminology
 - "Testing capabilities" in checklist paper
 - "Stress testing" in some others (but stress testing has a very different meaning in software testing: robustness to overload)
- Software engineering concepts translate, but names not adopted in ML community
 - specification-based testing, black-box testing
 - equivalence class testing, boundary-value analysis



Testing Invariants with Unlabeled Data



Randomly Generating "Realistic" Inputs is Possible

```
@Test
void testNextDate() {
    nextDate(2010, 8, 20)
    nextDate(2024, 7, 15)
    nextDate(2011, 10, 27)
    nextDate(2024, 5, 4)
    nextDate(2013, 8, 27)
    nextDate(2010, 2, 30)
}
```

But how do we know whether the computation is correct?

Automated Model Validation Data Generation?

```
@Test
void testCancerPrediction() {
    cancerModel.predict(generateRandomImage())
    cancerModel.predict(generateRandomImage())
    cancerModel.predict(generateRandomImage())
}
```

- But how do we get labels?

The Oracle Problem

```
assertEquals(??, factorPrime(15485863));
```



How do we know the expected output of a test?

Manually constructing outputs

```
@Test
void testNextDate() {
    assert nextDate(2010, 8, 20) == (2010, 8, 21);
    assert nextDate(2024, 7, 15) == (2024, 7, 16);
    assert nextDate(2010, 2, 30) throws InvalidInputException;
}
```

```
@Test
void testCancerPrediction() {
    assert cancerModel.predict(loadImage("random1.jpg")) == true;
    assert cancerModel.predict(loadImage("random2.jpg")) == true;
    assert cancerModel.predict(loadImage("random3.jpg")) == false;
}
```

Compare against reference impl.

```
@Test
void testNextDate() {
    assert nextDate(2010, 8, 20) == referenceLib.nextDate(2010, 8, 20);
    assert nextDate(2024, 7, 15) == referenceLib.nextDate(2024, 7, 15);
    assert nextDate(2010, 2, 30) == referenceLib.nextDate(2010, 2, 30)
}
```

```
@Test
void testCancerPrediction() {
    assert cancerModel.predict(loadImage("random1.jpg")) == ???;
}
```

assuming we have a correct implementation. *no reference implementation for ML problems*

Checking global specifications

Ensure no computation crashes

```
@Test
void testNextDate() {
    nextDate(2010, 8, 20)
    nextDate(2024, 7, 15)
}
```

```
@Test
void testCancerPrediction() {
    cancerModel.predict(generateRandomImage())
    cancerModel.predict(generateRandomImage())
}
```

Invariants as partial specification

```
class Stack {  
    int size = 0;  
    int MAX_SIZE = 100;  
    String[] data = new String[MAX_SIZE];  
    // class invariant checked before and after every method  
    private void check() {  
        assert(size >= 0 && size <= MAX_SIZE);  
    }  
    public void push(String v) {  
        check();  
        if (size < MAX_SIZE)  
            data[+size] = v;  
        check();  
    }  
    public void pop(String v) { check(); ... }
```

Invariants in Machine Learned Models?



Invariants in Large Language Models?



Invariants in Agentic Systems?



Invariants in Machine Learned Models (Metamorphic Testing)

Exploit relationships between inputs

- If two inputs differ only in **X** -> output should be the same
- If inputs differ in **Y** output should be flipped
- If inputs differ only in feature **F**, prediction for input with higher **F** should be higher
- ...

Metamorphic Testing, more formally

Formal description of relationships among inputs and outputs
(*Metamorphic Relations*)

In general, for a model f and inputs x define two functions to transform inputs and outputs g_I and g_O such that:

$$\forall x. f(g_I(x)) = g_O(f(x))$$

e.g. $g_I(x) = \text{replace}(x, \text{" is "}, \text{" is not "})$ and $g_O(x) = \neg x$

Some Capabilities are Invariants

Some capability tests can be expressed as invariants and automatically encoded as transformations to existing test data

- Negation should flip sentiment analysis result
- Typos should not affect sentiment analysis result
- Changes to locations or names should not affect sentiment analysis results

Robust.	INV: Add randomly generated URLs and handles to tweets	9.6	13.4	24.8	11.4	7.4	@JetBlue that selfie was extreme. @pi9QDK INV @united stuck because staff took a break? Not happy 1K.... https://t.co/PWK1jb INV
	INV: Swap one character with its neighbor (typo)	5.6	10.2	10.4	5.2	3.8	@JetBlue → @JeBtlue I cri INV @SouthwestAir no thanks → thakns INV
NER	INV: Switching locations should not change predictions	7.0	20.8	14.8	7.6	6.4	@JetBlue I want you guys to be the first to fly to # Cuba → Canada... INV @VirginAmerica I miss the #nerdbird in San Jose → Denver INV
	INV: Switching person names should not change predictions	2.4	15.1	9.1	6.6	2.4	...Airport agents were horrendous. Sharon → Erin was your saviour INV @united 8602947, Jon → Sean at http://t.co/58tuTgli0D , thanks. INV

Some Capabilities are Invariants

INV: Replace neutral words with other neutral words	9.4	16.2	12.4	10.2	10.2	@Virgin should I be concerned that → when I'm about to fly ... INV
						@united the → our nightmare continues... INV
DIR: Add positive phrases, fails if sent. goes down by > 0.1	12.6	12.4	1.4	0.2	10.2	@SouthwestAir Great trip on 2672 yesterday... You are extraordinary. ↑
						@AmericanAir AA45 ... JFK to LAS. You are brilliant. ↑
DIR: Add negative phrases, fails if sent. goes up by > 0.1	0.8	34.6	5.0	0.0	13.2	@USAirways your service sucks. You are lame. ↓
						@JetBlue all day. I abhor you. ↓

- For those that output should change: in ML we calculate the prediction probability
 - Add negation to positive sentences should decrease the probability of model predicting it to be a positive sentence

Examples of Invariants

Credit rating should not depend on gender:

- $\forall x. f(x[\text{gender} \leftarrow \text{male}]) = f(x[\text{gender} \leftarrow \text{female}])$

Synonyms should not change the sentiment of text:

- $\forall x. f(x) = f(\text{replace}(x, \text{"is not"}, \text{"isn't"}))$

Negation should swap meaning:

- $\forall x \in \text{"X is Y"}. f(x) = 1 - f(\text{replace}(x, \text{" is "}, \text{" is not "}))$

Robustness around training data:

- $\forall x \in \text{training data}. \forall y \in \text{mutate}(x, \delta). f(x) = f(y)$

Low credit scores should never get a loan (sufficient conditions for classification, "anchors"):

- $\forall x. x.\text{score} < 649 \Rightarrow \neg f(x)$

Identifying invariants requires domain knowledge of the problem!

Powerful, if we have this we have automatic test in production

On Testing with Invariants/Assertions

- Defining good metamorphic relations requires knowledge of the problem domain
- Good metamorphic relations focus on parts of the system
- Invariants usually cover only one aspect of correctness -- maybe capabilities
- Invariants and near-invariants can be mined automatically from sample data (see *specification mining* and *anchors*)

Approaches for Checking Invariants

- Generating test data (random, distributions) usually easy
- Transformations of existing test data
- Adversarial learning: For many techniques gradient-based techniques to search for invariant violations -- that's roughly analogous to symbolic execution in SE
- Early work on formally verifying invariants for certain models (e.g., small deep neural networks)

Summary: Invariants and Generation

- Generating sample inputs is easy, but knowing corresponding outputs is hard (oracle problem)
- Crashing bugs are not a concern
- Invariants + generated data can check capabilities or properties (metamorphic testing)
 - Inputs can be generated realistically or to find violations (adversarial learning)
- If inputs can be computed from outputs, tests can be automated (simulation-based testing)

Property-based Testing

How to Test Models for Generation Task?

- So far most of model testing techniques we discuss expect a "ground-truth" output.
- But what about "Generative AI", where no single ground-truth is available?
 - E.g., an LLM-based quiz maker

"Your goal is to create a well crafted set of four concise answers for a test for a specific question."

Property-based Testing

```
def test_gcd(): # greatest common divisor
    assert 1 == gcd(15, 7)
    assert 5 == gcd(15, 5)
    assert 3 == gcd(-9, 15)
```

```
def test_gcd(n, m):
    d = gcd(n, m)

    assert d > 0 # 1) `d` is positive
    assert n % d == 0 # 2) `d` divides `n`
    assert m % d == 0 # 3) `d` divides `m`

    # 4) no other number larger than `d` divides both `n` and `m`
    for i in range(d + 1, min(n, m)):
        assert (n % i) or (m % i)
```

Instead of writing unit tests, we can test properties a function should satisfy.

Property-based Testing For Models

"Your goal is to create a well crafted set of four concise answers for a test for a specific question."

- There are only four answers
- Each generated answer should contain <15 words
- All answers should be relevant to the question
- All answers should be covered in the course material (provided as context)

(Capabilities: what can the model do? Properties: what constraint should any output satisfy?)

Some properties are easier to test with programs, but how to test "vaguer" properties?

Testing Properties with LLMs

How to test "vaguer" properties? Use LLM as a judge!

- All answers should be relevant to the question
 - "Is the answer relevant to the question? Answer yes or no."

Need to make sure LLM judgments align with human judgments --
This is a common pitfall for practitioners.

- LLMs prefer verbose responses, similar to what they would generate, sensitive to option ordering.
- Measure alignment between judge verdicts and human verdicts on a sample
- A judge that agrees with humans 70% of the time can still be useful for catching regressions at scale

LLM-as-judge: prompt specifies criteria, provides context, requests verdict.

Common patterns:

- Binary pass/fail. "Does this response contain hallucinated facts? Answer yes or no."
- Scoring. "Rate the helpfulness of this response from 1-5."
- Pairwise comparison. "Which response better answers the user's question, A or B?"

Practical considerations: Cost, latency, reproducibility.

When to use what. Use programmatic checks when possible, LLM judges for subjective criteria, humans for high-stakes validation.

Breakout Discussion

Write a few ($n \geq 3$) property-based tests for the following prompt:

"Your task is to analyze project handouts. Summarize the handouts in a concise and clear manner, identifying key project goals, milestones, and risks. Output your findings as a short memo I can send to my team. The goal of the memo is to ensure my team stays aligned on the project's objectives, timelines, and deliverables while also identifying potential bottlenecks or challenges that could impact progress. Make sure to include all relevant details in your summary and analysis."

As a group, discuss what property-based tests you would write and how you would execute them.

Post answer to #1lecture tagging group members in Slack using template:

Property Test: ...

Test method: ...

Test prompt / program: ...

Other Testing Concepts

Simulation-Based Testing

In some cases it is easy to go from outputs to inputs:

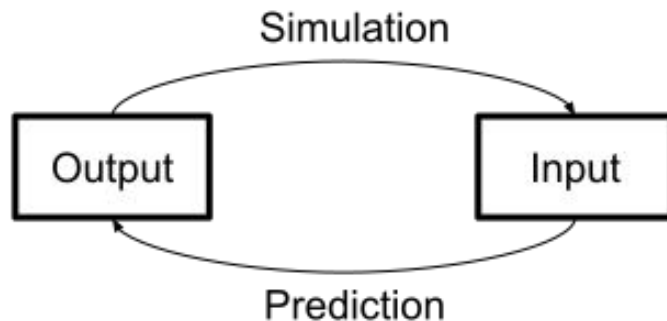
```
assertEquals(??, factorPrime(15485862));
```

```
randomNumbers = [2, 3, 7, 7, 52673]  
assertEquals(randomNumbers,  
             factorPrime(multiply(randomNumbers)));
```

Similar idea in machine-learning problems?

Simulation-Based Testing

- Derive input-output pairs from simulation, esp. in vision systems
- Example: Vision for self-driving cars:
 - Render scene -> add noise -> recognize -> compare recognized result with simulator state
- Quality depends on quality of simulator:
 - examples: render picture/video, synthesize speech, ...
 - Less suitable where input-output relationship unknown, e.g., cancer prognosis, housing price prediction



Further readings: Zhang, Mengshi, Yuqun Zhang, Lingming Zhang, Cong Liu, and Sarfraz Khurshid. "DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems." In Proc. ASE. 2018.

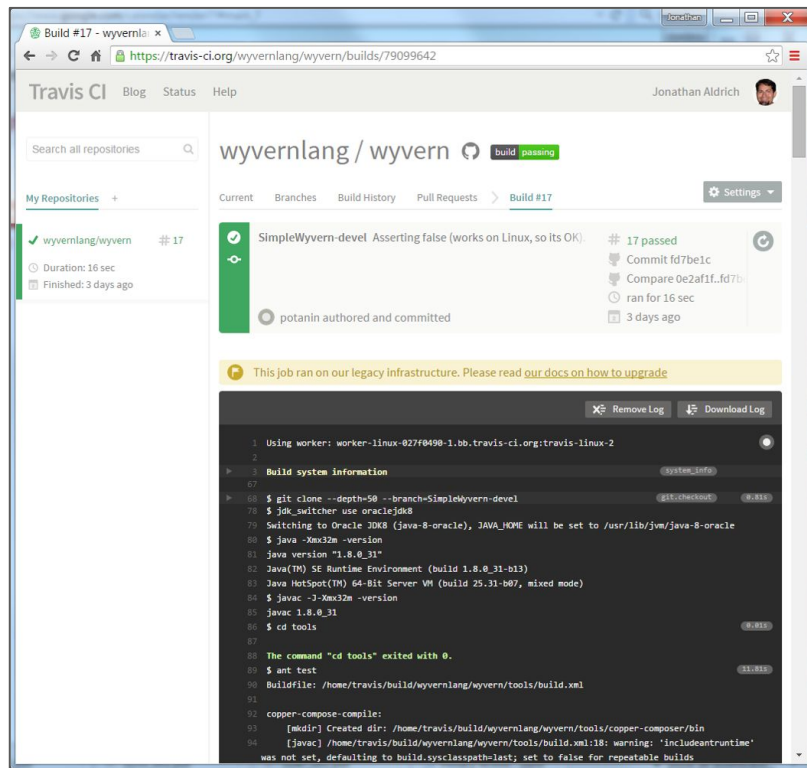
Structural Testing ("white box")

- Test case creation is driven by the implementation, not the specification.
 - Typically aiming to increase coverage.
 - Automated test generation similarly maximizing coverage (or finding crashes)
- Several ML coverage metrics have been proposed
 - All path of a decision tree?
 - All neurons activated at least once in a DNN? (several papers "neuron coverage")
 - Linear regression models??
- Often create artificial inputs, not realistic for distribution
- Adversarial learning techniques usually more efficient at finding invariant violations.

Regression Testing

- Whenever bug detected and fixed, add a test case
- Make sure the bug is not reintroduced later
- Execute test suite after changes to detect regressions
 - Ideally automatically with continuous integration tools
- Maps well to curating test **sets** for important populations in ML

Continuous Integration



Continuous Integration for Model Quality

- Testing script
 - Existing model: Automatically evaluate model on labeled training set; multiple separate evaluation sets possible, e.g., for slicing, regressions
 - Training model: Automatically train and evaluate model, possibly using cross-validation; many ML libraries provide built-in support
 - Report accuracy, recall, etc. in console output or log files
 - May deploy learning and evaluation tasks to cloud services
 - Optionally: Fail test below bound (e.g., accuracy $< .9$; accuracy $<$ last accuracy)
- Version control test data, model and test scripts, ideally also learning data and learning code (feature extraction, modeling, ...)
- Continuous integration tool can trigger test script and parse output, plot for comparisons (e.g., similar to performance tests)
- Optionally: Continuous deployment to production server

Summary

Curating test data

- Analyzing specifications, capabilities
- Not all inputs are equal: Identify important inputs (inspiration from specification-based testing)
- Slice data for evaluation
- Identifying capabilities and generating relevant tests

Automated random testing

- Feasible with invariants (e.g. metamorphic relations)
- Sometimes possible with simulation

Automate the test execution with continuous integration

Further readings

- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. "[Semantically equivalent adversarial rules for debugging NLP models](#)." In Proc. ACL, pp. 856-865. 2018.
- Barash, Guy, Eitan Farchi, Ilan Jayaraman, Orna Raz, Rachel Tzoref-Brill, and Marcel Zalmanovici. "[Bridging the gap between ML solutions and their business requirements using feature interactions](#)." In Proc. FSE, pp. 1048-1058. 2019.
- Ashmore, Rob, Radu Calinescu, and Colin Paterson. "[Assuring the machine learning lifecycle: Desiderata, methods, and challenges](#)." arXiv preprint arXiv:1905.04223. 2019.
- Christian Kaestner. [Rediscovering Unit Testing: Testing Capabilities of ML Models](#). Toward Data Science, 2021.
- D'Amour, Alexander, Katherine Heller, Dan Moldovan, Ben Adlam, Babak Alipanahi, Alex Beutel, Christina Chen et al. "[Underspecification presents challenges for credibility in modern machine learning](#)." *arXiv preprint arXiv:2011.03395* (2020).
- Segura, Sergio, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. "[A survey on metamorphic testing](#)." IEEE Transactions on software engineering 42, no. 9 (2016): 805-824.